Lab Assignment 8 and 9: SHA-512 & HMAC and File Integrity Verification

Course: M.Sc. Information Security
Level: Postgraduate

Objective:

1. To understand and implement SHA-512 hashing.
2. To implement HMAC using a shared secret key.
3. To verify file integrity using hashing algorithms.
4. To detect tampering in files.

Tools Required:

- Python 3.x
- hashlib and hmac libraries
- Any text file for testing

Task 8: SHA-512 Hash and HMAC Implementation

1. Write a Python program to compute SHA-512 hash of a file.
2. Display the hash value in hexadecimal format.
3. Generate a secret key and compute HMAC using SHA-512.
4. Modify the file and recompute hash and HMAC.
5. Compare results and observe changes.

Sample Code (Task 8):

```
import hashlib, hmac
key = b'secret_key'
with open('file.txt','rb') as f:
   data = f.read()
hash_val = hashlib.sha512(data).hexdigest()
hmac_val = hmac.new(key, data, hashlib.sha512).hexdigest()
print('SHA-512:', hash_val)
print('HMAC:', hmac_val)
```

Code

```
import hashlib

import hmac



# Create a sample file

with open("file.txt", "w") as f:

    f.write("Hello! This is a sample file.\nStudent ID:
12345\nCourse: Cryptography")


print("Created file.txt")

print("Content:", open("file.txt").read())

print()



# Compute SHA-512 hash of the file

with open("file.txt", "rb") as f:

    data = f.read()


hash_val = hashlib.sha512(data).hexdigest()


print("=" * 60)

print("ORIGINAL FILE")

print("=" * 60)
```

```
print("SHA-512 Hash:")
print(hash_val)
print()



# Generate a secret key and compute HMAC using SHA-512
secret_key = b"my_secret_key_123"
hmac_val = hmac.new(secret_key, data, hashlib.sha512).hexdigest()

print("Secret Key:", secret_key)
print()
print("HMAC-SHA512:")
print(hmac_val)
print()

# Modify the file and recompute hash and HMAC
with open("file.txt", "a") as f:
    f.write("\nThis line was added!")  # small change to the file

print("=" * 60)
print("AFTER MODIFYING FILE")
print("=" * 60)

with open("file.txt", "rb") as f:
```

```
    modified_data = f.read()


new_hash = hashlib.sha512(modified_data).hexdigest()

new_hmac = hmac.new(secret_key, modified_data,
hashlib.sha512).hexdigest()


print("New SHA-512 Hash:")

print(new_hash)

print()

print("New HMAC-SHA512:")

print(new_hmac)

print()


# Compare results

print("=" * 60)

print("COMPARISON")

print("=" * 60)

print(f"Hash changed : {hash_val != new_hash}")

print(f"HMAC changed : {hmac_val != new_hmac}")

print()

print(f"Original Hash (first 40 chars): {hash_val[:40]}...")

print(f"New Hash     (first 40 chars): {new_hash[:40]}...")

print()

print(f"Original HMAC (first 40 chars): {hmac_val[:40]}...")

print(f"New HMAC      (first 40 chars): {new_hmac[:40]}...")
```

Output

```
C:\Users\chirag\Desktop\code\infosec\sha512>py sha512.py
Created file.txt
Content: Hello! This is a sample file.
Student ID: 12345
Course: Cryptography

============================================================
ORIGINAL FILE
============================================================
SHA-512 Hash:
c8a8a250dcce5986f36c1c6c3c55fa353cea40e688897a7ed8177e8be5f85066cda1c07947f5da
05b1962a7740

Secret Key: b'my_secret_key_123'

HMAC-SHA512:
57299c51a82ff37df18fd2abe5c4277e343298b51336a7a328226fc0defa1c253b5afc5c98f350
2550a5550c5c

============================================================
AFTER MODIFYING FILE
============================================================
New SHA-512 Hash:
ea119a49c000f9c8c16013b0c4342164eff5c810de71606667c990c7f3668930c1773a11ff6b7d
21231ba60484

New HMAC-SHA512:
be0a657993f6f8f128566844d18b665a390d0b4f4b0adc8efe5abdb226339add7002b6910a708d
ef468fba7543

============================================================
COMPARISON
============================================================
Hash changed : True
HMAC changed : True

Original Hash (first 40 chars): c8a8a250dcce5986f36c1c6c3c55fa353cea40e6...
New Hash     (first 40 chars): ea119a49c000f9c8c16013b0c4342164eff5c810...

Original HMAC (first 40 chars): 57299c51a82ff37df18fd2abe5c4277e343298b5...
New HMAC      (first 40 chars): be0a657993f6f8f128566844d18b665a390d0b4f...
```

C:\Users\chirag\Desktop\code\infosec\sha512\file.txt (active) - Sublime Text (UNREGISTERED)

integrity_check.py | file.txt — integrity_verification | file.txt — sha512

```
1  Hello! This is a sample file.
2  Student ID: 12345
3  Course: Cryptography
4  This line was added!
```

Task 9: File Integrity Verification

1. Store original hash of a file.
2. Recompute hash after some time.
3. Compare hashes.
4. Display 'Integrity Verified' if hashes match.
5. Display 'Integrity Compromised' otherwise.

Sample Code (Task 9):

```
import hashlib
def get_hash(file):
   with open(file,'rb') as f:
      return hashlib.sha512(f.read()).hexdigest()

original = get_hash('file.txt')
current = get_hash('file.txt')

if original == current:
   print('Integrity Verified')
else:
   print('Integrity Compromised')
```

Expected Output:

- SHA-512 hash values
- HMAC values
- Integrity verification result

Code

```
import hashlib

import json

import os
```

```
# Helper function to compute SHA-512 hash of a file

def get_hash(filename):

    with open(filename, "rb") as f:

        return hashlib.sha512(f.read()).hexdigest()




# Create a file and store its original hash

with open("file.txt", "w") as f:

    f.write("Hello! This is a sample file.\nStudent ID:
12345\nCourse: Cryptography")


original_hash = get_hash("file.txt")


# Save the hash to a file (simulates storing it for later
comparison)

with open("stored_hash.txt", "w") as f:

    f.write(original_hash)


print("File created: file.txt")

print("Original hash stored in stored_hash.txt")

print()

print("Original SHA-512 Hash:")

print(original_hash)

print()
```

```python
# Check integrity WITHOUT modifying the file
print("=" * 60)
print("CHECK 1: File not modified")
print("=" * 60)

stored_hash = open("stored_hash.txt").read()
current_hash = get_hash("file.txt")

print("Stored Hash :", stored_hash[:60], "...")
print("Current Hash:", current_hash[:60], "...")
print()

if current_hash == stored_hash:
    print("Result: Integrity Verified ✔")
else:
    print("Result: Integrity Compromised ✘")


# Modify the file and recheck integrity
print()
print("=" * 60)
print("CHECK 2: File modified")
print("=" * 60)
```

```
with open("file.txt", "a") as f:
    f.write("\nThis line was secretly added!")  # tamper with the file

print("File has been modified!")
print()

current_hash = get_hash("file.txt")

print("Stored Hash :", stored_hash[:60], "...")
print("Current Hash:", current_hash[:60], "...")
print()

if current_hash == stored_hash:
    print("Result: Integrity Verified ✔")
else:
    print("Result: Integrity Compromised ✘")
```

Output

```
C:\Users\chirag\Desktop\code\infosec\integrity_verification>py integrity_check.py
File created: file.txt
Original hash stored in stored_hash.txt

Original SHA-512 Hash:
c8a8a250dcce5986f36c1c6c3c55fa353cea40e688897a7ed8177e8be5f85066cda1c07947f5da2ea8
05b1962a7740

============================================================
CHECK 1: File not modified
============================================================
Stored Hash : c8a8a250dcce5986f36c1c6c3c55fa353cea40e688897a7ed8177e8be5f8 ...
Current Hash: c8a8a250dcce5986f36c1c6c3c55fa353cea40e688897a7ed8177e8be5f8 ...

Result: Integrity Verified ✔

============================================================
CHECK 2: File modified
============================================================
File has been modified!

Stored Hash : c8a8a250dcce5986f36c1c6c3c55fa353cea40e688897a7ed8177e8be5f8 ...
Current Hash: c44228aa16a782afd4cb3835e190d5b8437f7cc9687d3677c76b9c501c19 ...

Result: Integrity Compromised ✗

C:\Users\chirag\Desktop\code\infosec\integrity_verification>
```

```
C:\Users\chirag\Desktop\code\infosec\integrity_verification\file.txt (
integrity_check.py        file.txt
1  Hello! This is a sample file.
2  Student ID: 12345
3  Course: Cryptography
4  This line was secretly added!
```

Viva Questions:

1. What is SHA-512?

Ans: SHA-512(Secure Hash Algorithm 512) is a cryptographic hash function from the SHA-2 family. It has the following properties:

- Takes any size input
- Produces a fixed sized output
- Deterministic and one-way
- Hard to reverse
- Hard to find two inputs with the same hash
- Small change in input produce large change in output(Avalanche effect)

2. Difference between hash and HMAC?

Ans:

Hash	**HMAC**
No key is used	A secret key is used
Used to maintain data integrity.	Used for data integrity and authentication.
Example: SHA-512	Example: HMAC_SHA512
Anyone to generate hash.	Only those with the key can generate hash

3. Why is HMAC more secure?

Ans: HMAC is more secure because it uses a secret key. It ensures that only someone with the secret key can generate the hash or verify it. It prevents tampering of data and impersonation. HMAC also protects against length extension attacks.

4. What is integrity in security?

Ans: Integrity means that the data has not been altered. It ensures that the data that was received is the same as the data which was sent. It is crucial in applications such as file downloads, message transmission, etc. Data integrity can be verified by comparing the HASH or using HMAC.

5. Can hashing alone ensure authenticity?

Ans: No, Hashing alone can't ensure authenticity because hashing has no secret key, which means anyone can compute the hash of the data. An attacker can modify the data, recompute the hash of the modified data, and send both.

To ensure authenticity, we need to use HMAC or digital signatures(public/private key).